

METU Computer Engineering

CEng 111 – Fall 2026
Week 3a

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

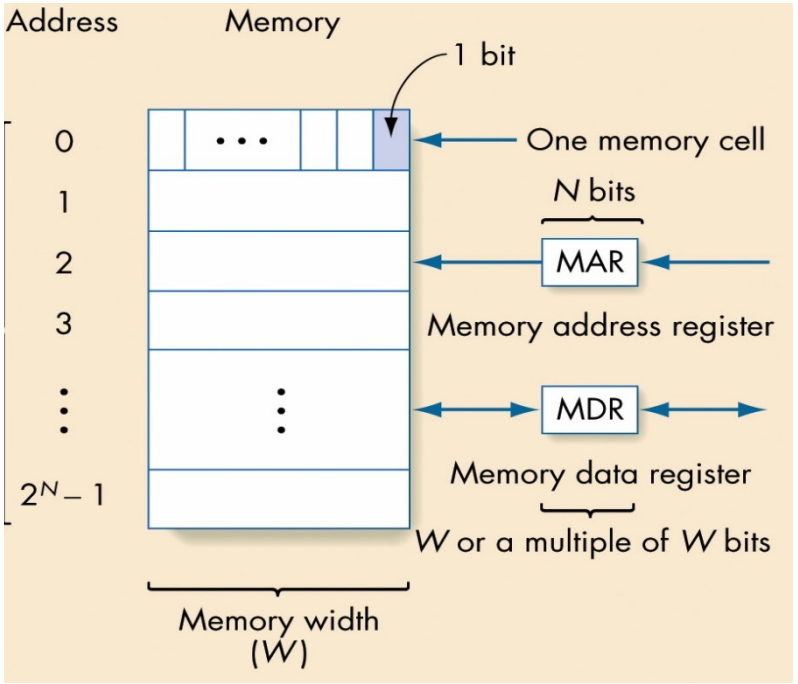
Memory and Cache (continued)

Previously on CENG111!

- RAM (Random Access Memory)

  Often called *memory*, *primary memory*

  - Memory made of addressable "cells"
  - Cell size is 8 bits
    - Nowadays, it is 32 or 64 bits.
  - All memory cells accessed in equal time
  - Memory address
    - Unsigned binary number with N bits
    - Address space is then 2^N cells



Memory and Cache (continued)

Previously on CENG111!

- Parts of the memory subsystem
  - Fetch/store (or Read/Write) controller
    - Fetch: retrieve a value from memory
    - Store: store a value into memory
  - Memory address register (MAR)
  - Memory data register (MDR)



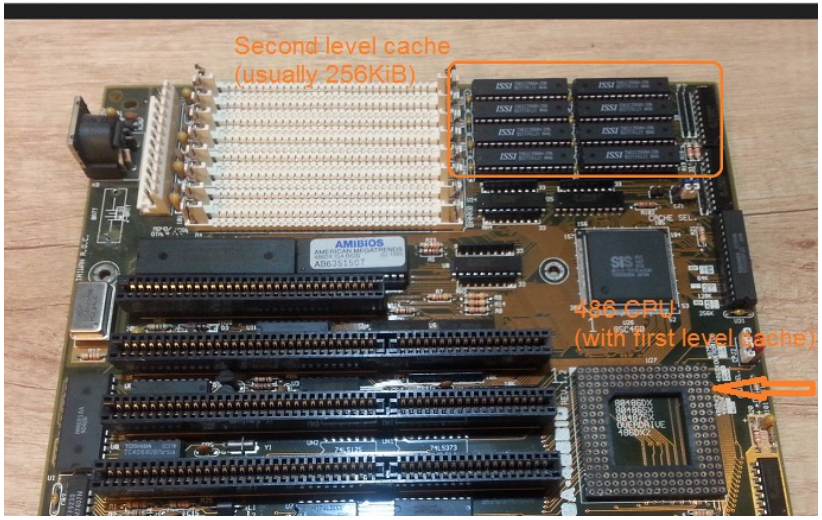
METU Computer Engineering

Previously on CENG111!

Cache Memory

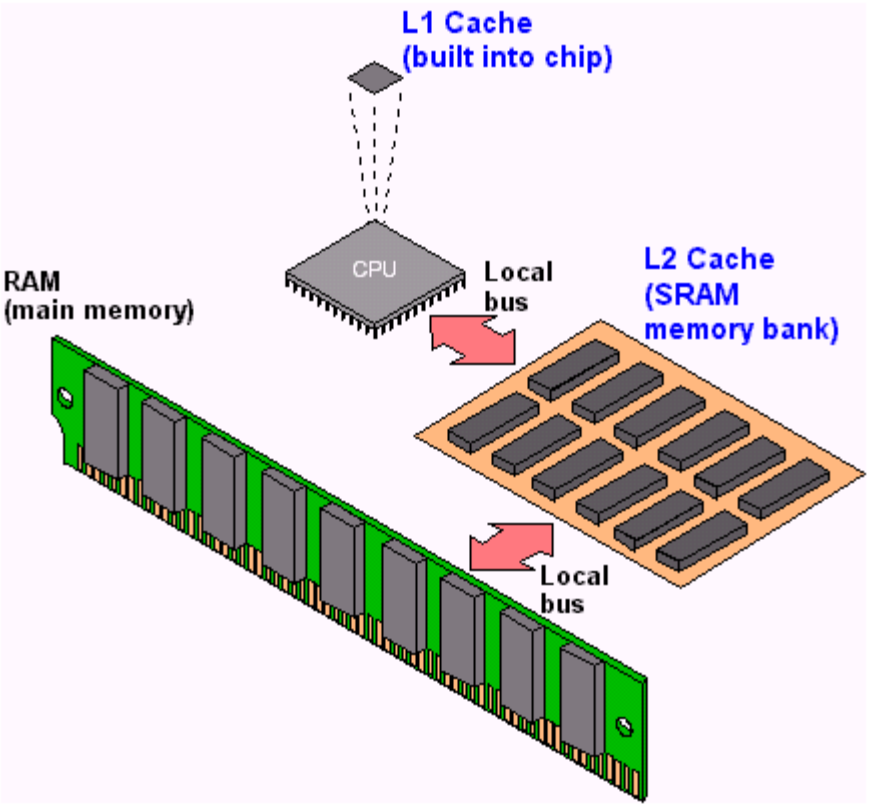
- Memory access is much slower than processing time
- Faster memory is too expensive to use for all memory cells
- Locality principle
  - Once a value is used, it is likely to be used again
- Small size, fast memory just for values currently in use speeds computing time

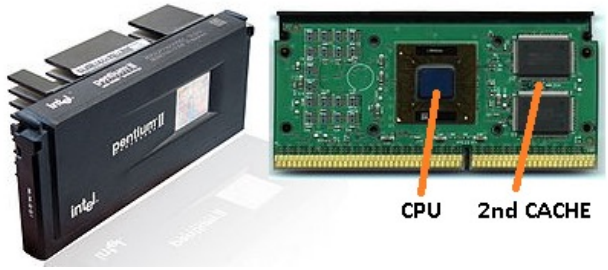
Previously on CENG111!

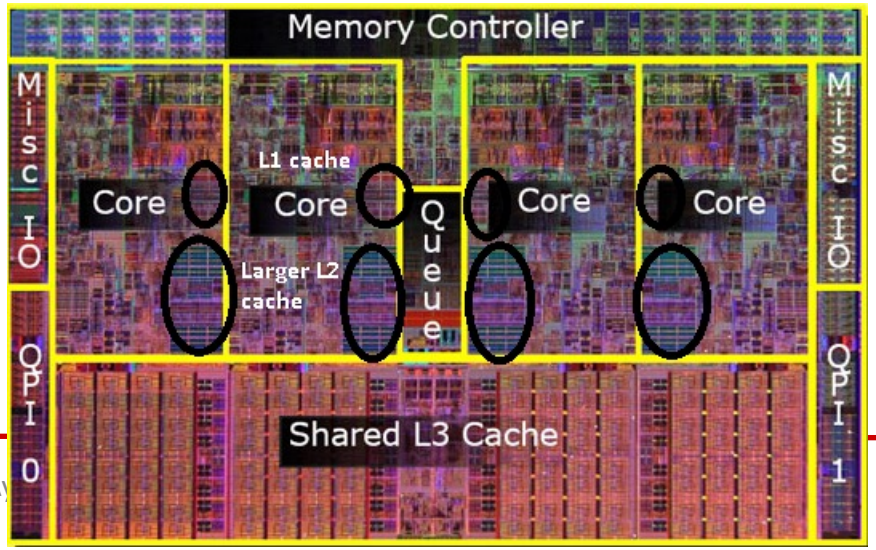


80486: (1989)

From Computer Desktop Encyclopedia
© 1999 The Computer Language Co. Inc.









Today

- Continue with the Von Neumann Architecture
  - CPU
    - Control Unit and the Fetch-Decode-Execute Cycle
    - Arithmetic Logic Unit

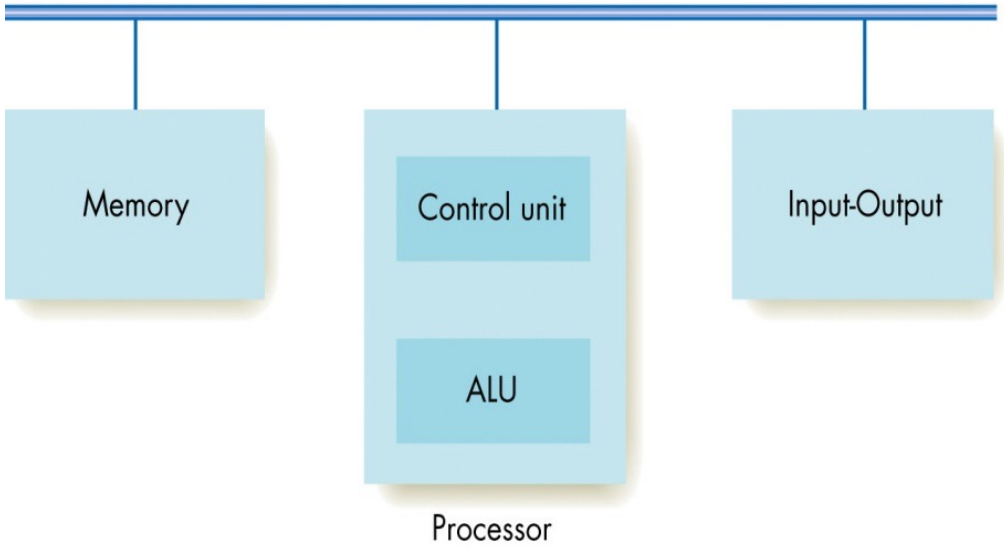


Administrative Notes

- History of computing quiz
  - Material shared
  - Online quiz:
    - Today (13 Oct) between 19:00 and 19:30
    - On ODTUClass page of the course.
- Labs are starting this week!
- Linux Workshop:
  - Today (13 Oct) 19:30

Central Processing Unit (CPU)

- Control Unit
- Arithmetic Logic Unit (ALU)



METU Computer Engineering

The Control Unit



- Manages stored program execution
- Task

1. Fetch from memory the next instruction to be executed
2. Decode it: determine what is to be done
3. Execute it: issue appropriate command to ALU, memory, and I/O controllers



Machine Language Instructions

- Can be decoded and executed by control unit
- Parts of instructions
  - Operation code (op code)
    - Unique unsigned-integer code assigned to each machine language operation
  - Address field(s)
    - Memory addresses of the values on which operation will work



Typical Machine Language Instruction Format

More on Instructions



- LOAD X -> Load register R with the contents of memory cell X
- STORE X -> Store register R into memory cell X
- MOVE X, Y -> Copy the contents of X into Y
- ADD X -> Add contents of X to the contents of R
- ADD X,Y -> Add contents of X to the contents of Y, and put the result in register R
- COMPARE X, Y-> Set GT (greater than), EQ (equal) and LT (less than) condition codes
- JUMP X-> Jump unconditionally to the instruction in cell X
- JUMPGT X-> Jump, if GT=1, to the instruction in cell X
- HALT



Machine Language Instructions (continued)

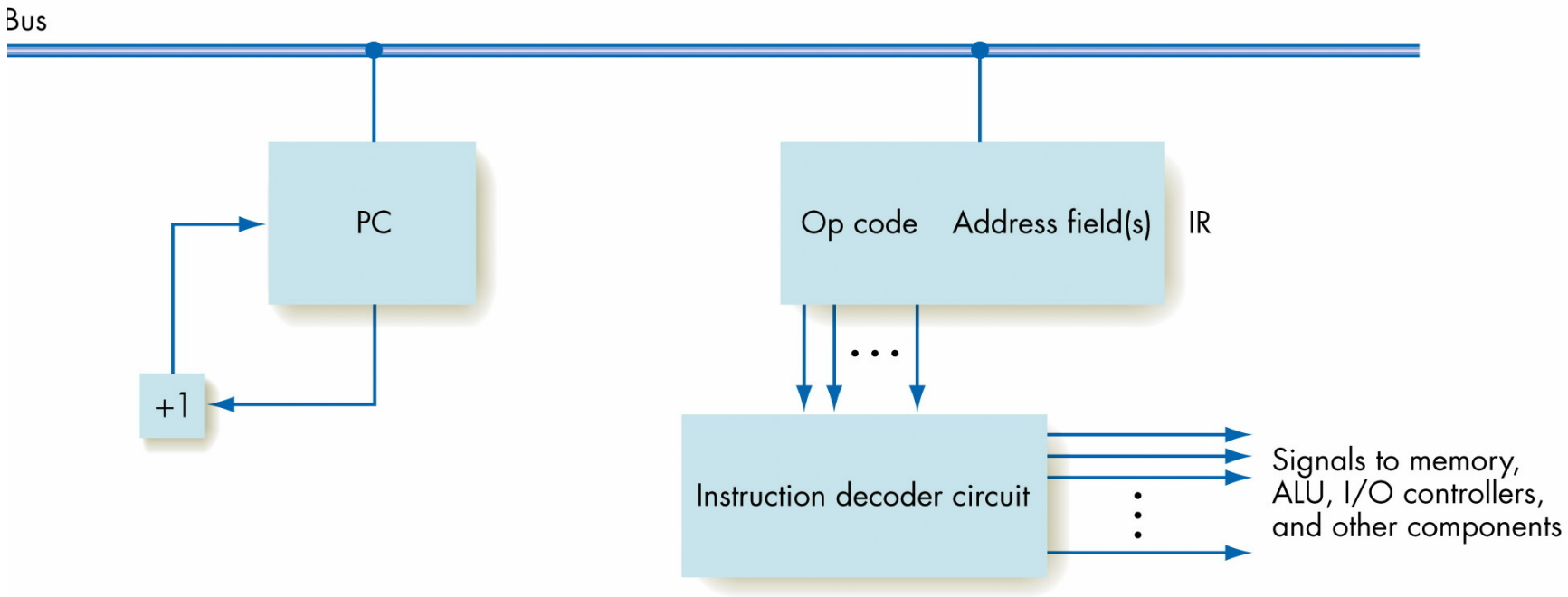
- Types of machine instructions:
  - Data transfer
    - Move values to and from memory and registers
  - Arithmetic/logic
    - Perform ALU operations that produce numeric values
  - Compares
    - Set bits of compare register to hold result
  - Branches
    - Jump to a new memory address to continue processing



Control Unit Registers and Circuits

METU Computer Engineering

- Parts of control unit
  - Linked to other subsystems
  - Instruction decoder circuit
  - Two special registers:
    - Program Counter (PC)
      - Stores the memory address of the next instruction to be executed
    - Instruction Register (IR)
      - Stores the code for the current instruction



Organization of the Control Unit Registers and Circuits

Address	Contents
100	Value of *a*
101	Value of *b*
102	Value of *c*

Algorithmic notation	Machine Language Instruction Sequences		
	Address	Contents	(Commentary)
		⋮	
1. Set *a* to the value *b* + *c*	50	LOAD 101	Put the value of *b* into register R.
	51	ADD 102	Add *c* to register R. It now holds *b* + *c*.
	52	STORE 100	Store the contents of register R into *a*.

Address	Contents
100	Value of *a*
101	Value of *b*
102	Value of *c*

Algorithmic notation	Address	Machine Language Instruction Sequences Contents	(Commentary)
		⋮	
1. Set *a* to the value *b* + *c*	50	LOAD 101	Put the value of *b* into register R.
	51	ADD 102	Add *c* to register R. It now holds *b* + *c*.
	52	STORE 100	Store the contents of register R into *a*.
2. If *a* > *b* then	50	COMPARE 100, 101	Compare *a* and *b* and set condition codes.
set *c* to the value *a*	51	JUMPGT 54	Go to location 54 if *a* > *b*.
Else	52	MOVE 101, 102	Get here if *a* ≤ *b*, so move *b* into *c*
set *c* to the value *b*	53	JUMP 55	and skip the next instruction.
	54	MOVE 100, 102	Move *a* into *c*.
	55	• • •	Next statement begins here.